tree, or just the root of the branch; thus, if we end up in a case similar to figure 9.17 (either with missing left or right branch, indifferently), we might have to call `remove` recursively on `n-1` nodes, in the absolute worst case.

However, we assumed that our k-d tree is balanced. Under this assumption, left and right branches should have more or less the same number of nodes, and therefore if the right branch of a node is empty, the probability that left branch has 1 node is still high, that it has 2 nodes is already less likely, and it goes down with 3 nodes etc..., so that for a certain constant, for example 5, we can say that in a situation like the one in figure 9.17, where a node has a single branch, then it is highly unlikely that branch has more than a constant number of nodes (say 5). And we can thus assume that, in a balanced tree, such an imbalanced case can happen at most a constant number of times during a call to `remove`. Or, more precisely, on a large number of removals, we can assume that the amortized cost of running the code blocks starting at lines #4 and #8 would be `Tk(n/2)`.

Listing 9.8 The remove method

```
function remove(node,
point)                                                    #1
  if node == null
then                                                          #2
    return null
  elsif node.point == point
then                                                  #3
    if node.right != null
then                                                    #4
```